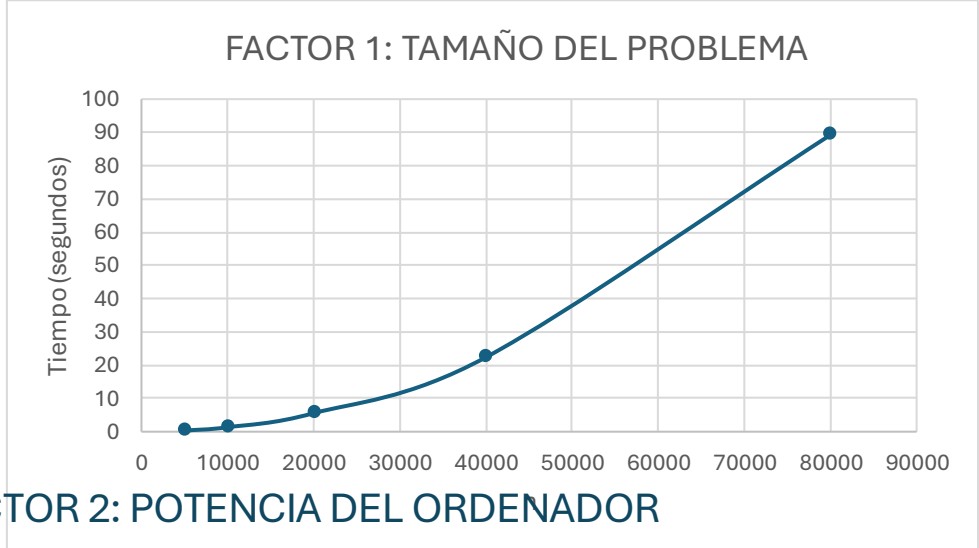
Práctica 0

FACTOR 1: TAMAÑO DEL PROBLEMA

n	5000	10000	20000	40000	80000
tiempo (segundos)	0,332	1,337	5,563	22,392	89,255

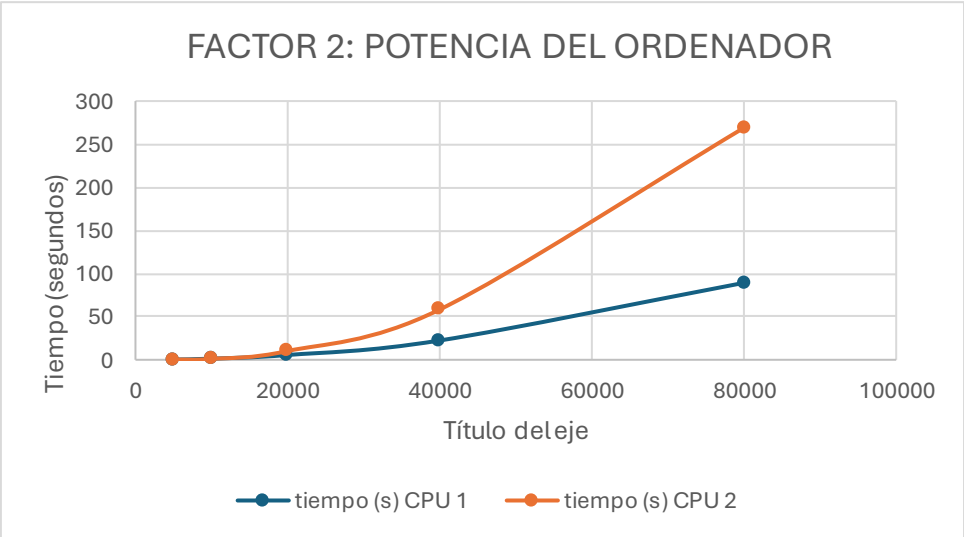


FACTOR 2: POTENCIA DEL ORDENADOR

CPU 1:

CPU 2: 13th Gen Intel(R) Core(TM) i7-1355U 1.70 GHz

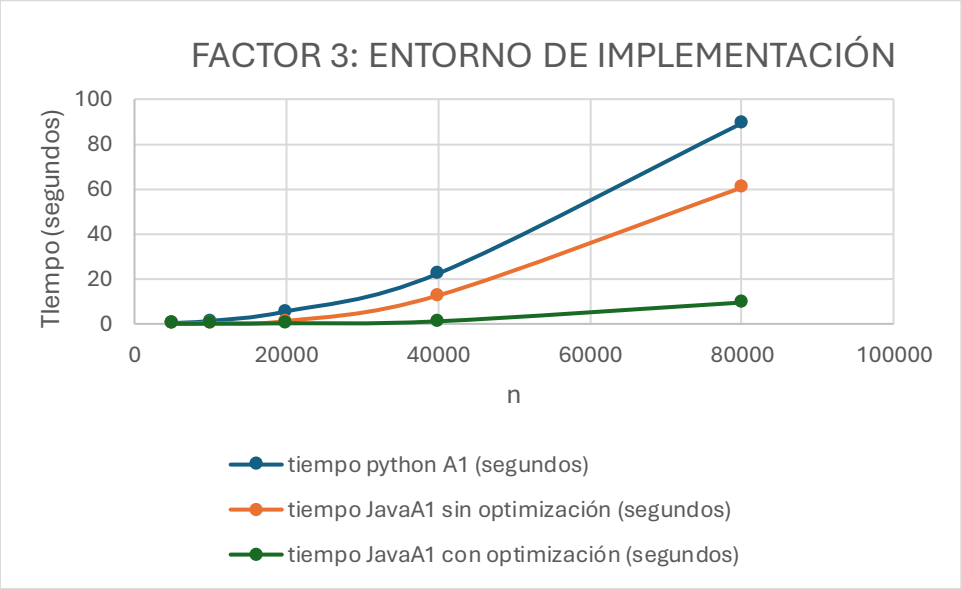
n	5000	10000	20000	40000	80000
tiempo (s) CPU 1	0,332	1,337	5,563	22,392	89,255
tiempo (s) CPU 2	0,252	1,035	10,171	58,337	269,07



FACTOR 3: ENTORNO DE IMPLEMENTACIÓN

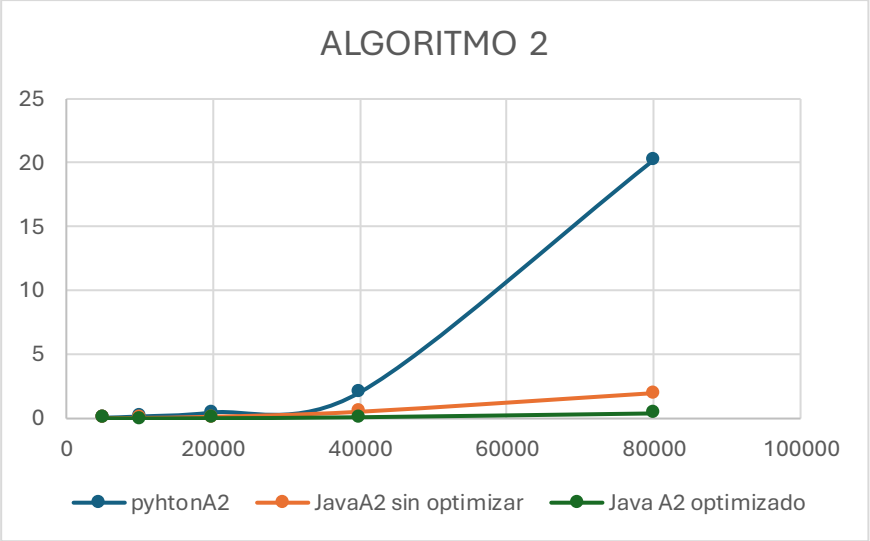
n	5000	10000	20000	40000	80000

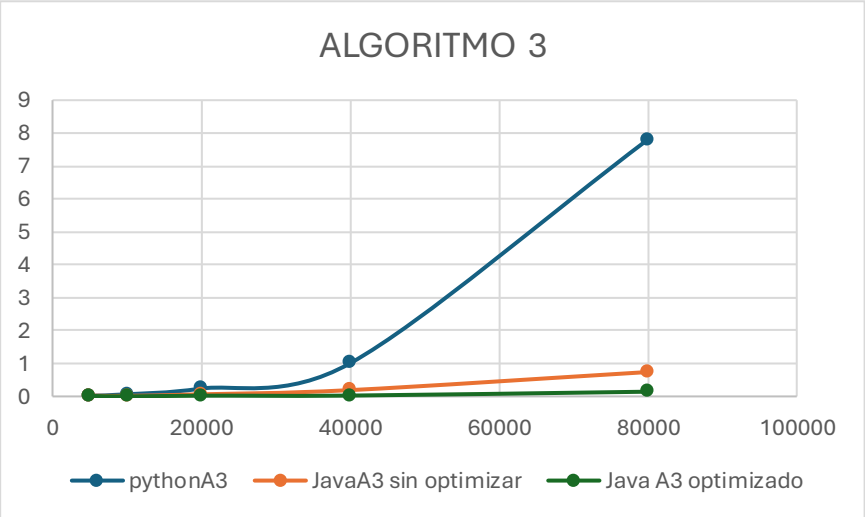
tiempo python A1 (segundos)	0,332	1,337	5,563	22,392	89,255
tiempo JavaA1 sin optimización (segundos)	0,093	0,361	1,27	12,643	60,634
tiempo JavaA1 con optimización (segundos)	0,044	0,059	0,266	1,145	9,509

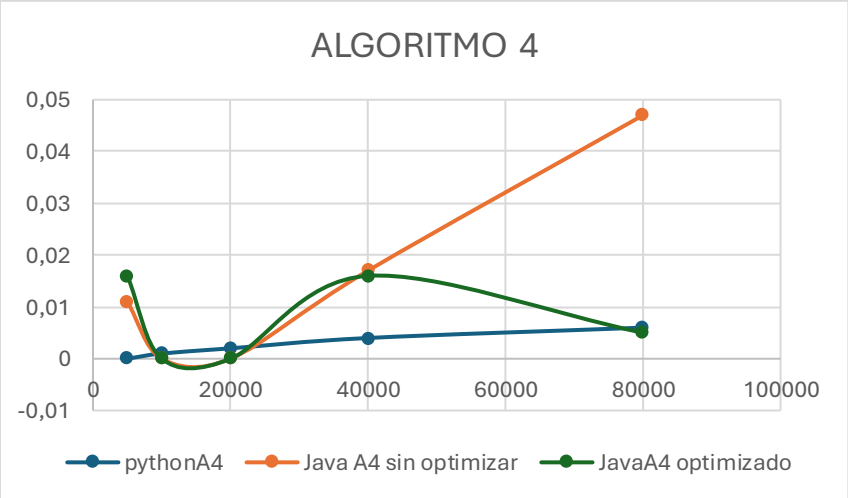


FACTOR 4: ALGORITMO EMPLEADO

n	5000	10000	20000	40000	80000
pyhtonA2	0,06	0,16	0,48	2,06	20,22
pythonA3	0,02	0,07	0,24	1,01	7,81
pythonA4	0,00	0,00	0,00	0,00	0,01
JavaA2 sin optimizar	0,03	0,05	0,14	0,53	1,99
Java A2 optimizado	0,03	0,01	0,03	0,10	0,40
JavaA3 sin optimizar	0,03	0,02	0,06	0,19	0,74
Java A3 optimizado	0,02	0,00	0,02	0,03	0,14
Java A4 sin optimizar	0,01	0,00	0.016	0,02	0,05
JavaA4 optimizado	0,02	0,00	0,00	0,02	0,01


ALGORITMO 2
25
20
15
10
5
0
0
20000
40000
60000
80000
100000
pyhtonA2
JavaA2 sin optimizar
Java A2 optimizado


ALGORITMO 3
9
8
7
6
5
4
3
2
1
0
0
20000
40000
60000
80000
100000
pythonA3
JavaA3 sin optimizar
Java A3 optimizado


ALGORITMO 4
0,05
0,04
0,03
0,02
0,01
0
-0,01
0
20000
40000
60000
80000
100000
pythonA4
Java A4 sin optimizar
JavaA4 optimizado

Practica1.1

1. **¿Calcular cuántos años más podremos seguir utilizando esta forma de contar? Explica el razonamiento seguido para realizar el cálculo.**
   Como el dato de milisegundos se almacena en un tipo long (64bits), tendremos que calcular cuántos números se pueden representar en 64 bits.
   Por lo tanto, serían 2^{64} números = 18.446.744.073.709.551.616 números.
   Teniendo en cuenta que la cuenta empezó el 1 de enero de 1970, calculamos en qué año tendremos que parar la cuenta porque ya no se puedan representar más números: $\frac{2^{64}}{3{,}154e{+}10} = 5{,}8 * 10^8$ años. Si a este resultado le restamos los años que ya lleva contándose, (2026-1970=56), tenemos que podremos seguir con este método unos 579.999.944 años.

Despreciaremos tiempos por debajo de 50 ms y mientras más grande sea el tiempo, más fiable (3578 es más fiable que 123).
Tabla de tiempos para el algoritmo Vector2:

n	1.000.000	10.000.000	50.000.000	100.000.000	200.000.000
Tiempo (ms)	5	48	239	481	958

2. **¿Por qué a veces el tiempo medido sale 0?**

   Porque la operación tarda menos de 1ms en realizarse, igual tarda 0,2ms pero se muestra como 0ms.

3. **¿A partir de qué tamaño de problema (n) empezamos a obtener tiempos fiables? ¿Por qué?**

   Empezamos a obtener tiempos fiables a partir de un valor de n superior a 10.000.000 porque salen valores de tiempo superiores a 50ms. Esto es así porque existen procesos internos del sistema que se ejecutan con mayor prioridad que el programa que estamos midiendo. Los milisegundos que tardan dichos procesos se añaden a los que tarda nuestro programa, por ello para valores de tiempo tan pequeños, la medida no resulta fiable por el error relativo.

```
n   tTiempo
10000   0
20000   0
40000   0
80000   0
160000  0
320000  2
640000  3
1280000 7
2560000 12
5120000 24
10240000        48
20480000        97
40960000        200
81920000        389

Fin de la medicion de tiempos *****
```

Resultado de ejecutar Vector3 con ordenador de clase.

Los tiempos resultantes al ejecutar Vector3 son demasiado bajos, para solucionar eso tenemos implementado Vector4 que repite el algoritmo un mismo número de veces para cada valor de n.

A partir de este punto las mediciones se realizaron con un ordenador tal que:
Procesador: 13th Gen Intel(R) Core(TM) i7-1355U 1.70 GHz.

RAM instalada: 16,0 GB

```
repeticiones = 1
n        Tiempo
10000    0
20000    0
40000    0
80000    5
160000   1
320000   10
640000   7
1280000  0
2560000  20
5120000  29
10240000         57
20480000         300
40960000         1000
81920000         1396

Fin de la medicion de tiempos *****
```

Toma de tiempos Vector4 con 1 repetición

```
repeticiones = 10
n        Tiempo
10000    1
20000    3
40000    2
80000    4
160000   8
320000   16
640000   34
1280000  60
2560000  126
5120000  248
10240000         1183
20480000         3277
40960000         6250
81920000         12569

Fin de la medicion de tiempos *****
```

Toma de tiempos Vector4 con 10 repeticiones

```
repeticiones = 100
n        Tiempo
10000    10
20000    11
40000    20
80000    39
160000   78
320000   159
640000   344
1280000  696
2560000  2015
5120000  9012
10240000         18007
20480000         35158
40960000         65884
81920000         130519

Fin de la medicion de tiempos *****
```

Toma de tiempos Vector4 con 100 reps

```
repeticiones = 1000
n        Tiempo
10000    48
20000    102
40000    212
80000    431
160000   880
320000   2680
640000   9786
1280000  19799
2560000  46185
5120000  89771
10240000         171147
20480000         329490
40960000         752507
81920000         2457242

Fin de la medicion de tiempos *****
```

Toma de tiempos Vector4 con 1000 reps

Escoger 1000 repeticiones para calcular los tiempos, ya que las medidas sobrepasan los 50ms en prácticamente todos los valores de n.

4. **¿Qué pasa con el tiempo si el tamaño del problema se multiplica por 2?**
   Como este algoritmo tiene una complejidad lineal O(n), entonces el tiempo sería el doble.
5. **¿Qué pasa con el tiempo si el tamaño del problema se multiplica por otro k que no sea 2? (Pruebe, por ejemplo, para k=3 y k=4 y compruebe los tiempos obtenidos.)**
   El tiempo se multiplicará por el valor de k, si es 3 se triplicará y si es 4 se cuatriplicará.

```
repeticiones = 1000
n        Tiempo
10000    47
30000    132
90000    425
270000   1181
810000   10826
2430000  38856
7290000  115069
21870000         354349
65610000         1006904

Fin de la medicion de tiempos *****
```

Toma de tiempos con k=3

```
repeticiones = 1000
n        Tiempo
10000    41
40000    179
160000   818
640000   6226
2560000  37158
10240000         155751
40960000         880678

Fin de la medicion de tiempos *****
```

Toma de tiempos con k=4

6. **¿Razone si los tiempos obtenidos son los que se esperaban de la complejidad lineal O(n)?**

Sí, ya que el tiempo sería T(k*n) = k*T(n). Que es lo que se muestra con las medidas, aproximadamente.

TABLA 1 (tiempo en ms sin optimización)

n	Tsuma	Tmáximo
10.000	0,048	0,080
20.000	0,182	0,189
40.000	0,212	0,277
80.000	0,431	0,607
160.000	0,880	1,136
320.000	2,680	7,482
640.000	9,786	14,055
1.280.000	19,799	35,617
2.560.000	46,185	59,459
5.120.000	89,771	131,213
10.240.000	171,147	229,671
20.480.000	329,490	404,076
40.960.000	752,507	1.008,132
81.920.000	2.457,242	3.088,429

TABLA 2 (tiempo en ms sin optimización)

n	Tcoincidencias1	Tcoincidencias2
10.000	575	0,063
20.000	2307	0,125
40.000	29.668	0,251
80.000	Fdt	0,518
160.000	Fdt	1,057
320.000	Fdt	7,503
640.000	Fdt	25,914
1.280.000	Fdt	40,306
2.560.000	Fdt	75,808
5.120.000	Fdt	163,795
10.240.000	Fdt	296,048
20.480.000	Fdt	645,789
40.960.000	Fdt	1.373,095
81.920.000	Fdt	2.337,076

Los tiempos obtenidos coinciden con los esperados ya que el algoritmo de suma tenía complejidad lineal y los tiempos se van incrementando linealmente, igual que el algoritmo maximo.
El algoritmo coincidencias1, en cambio, tiene una complejidad cuadrática lo que se observa en los tiempos que cada vez son más elevados llegando a sobrepasar el límite de 60 segundos. El algoritmo coincidencias2 es lineal, lo que se aprecia en los tiempos medidos.

Practica 1.2

Tbucle1 => complejidad cuasi-lineal O(n*logn)
Tbucle2 = > O(n^2*logn)

Tbucle3 => O(n^2*logn), tarda más que el anterior
Tbucle4 => O(n^3)
TABLA1 (tiempos en milisegundos y SIN_OPTIMIZACIÓN):

n	Tbucle1-1000	Tbucle2	Tbucle3	Tbucle4
100	0	1	1	1
200	0	2	4	8
400	0	5	15	46
800	0	22	73	326
1600	0	57	263	2.594
3200	1	238	1.130	20.565
6400	0	944	4.762	Fdt
12800	2	4.331	19.986	Fdt
25600	4	18.884	Fdt	Fdt
51200	8	74.867	Fdt	Fdt

TABLA2 (tiempos en milisegundos y SIN_OPTIMIZACIÓN):

n	Tbucle5	Tbucle6	Tbucle7
100	3	33	235
200	12	267	3.657
400	58	2.284	57.920
800	267	19.935	Fdt
1600	1263	166.553	Fdt
3200	5.847	Fdt	Fdt
6400	26.554	Fdt	Fdt

Los resultados concuerdan con lo esperado ya que Tbucle5 crece de manera prácticamente cuadrática, Tbucle6 crece casi de manera cúbica y Tbucle7 crece de acuerdo a su complejidad O(n^4)

TABLA3 (tiempos en milisegundos y SIN_OPTIMIZACIÓN):

n	TBucle1(t1)	TBucle2(t2)	T1/t2
100	0	1	0
200	0	2	0
400	0	5	0
800	0	22	0
1600	0	57	0
3200	1	238	0,00420168
6400	0	944	0
12800	2	4.331	0,00046179
25600	4	18.884	0,00021182
51200	8	74.867	0,00010686

TABLA 4 (tiempos en milisegundos y SIN_OPTIMIZACIÓN):

n	TBucle3(t1)	TBucle2(t2)	T1/t2
100	1	1	1

200	4	2	2
400	15	5	3
800	73	22	3,31818182
1600	263	57	4,61403509
3200	1.130	238	4,74789916
6400	4.762	944	5,04449153
12800	19.986	4.331	4,61463865
25600	Fdt	18.884	¿?
51200	Fdt	74.867	¿?

TABLA5 (tiempos en milisegundos):

n	Bucle4-python	Bucle4 Java SIN OPTIMIZACIÓN	tPython/tJava
100	2	1	2
200	20	8	2,5
400	160	46	3,47826087
800	1.378	326	4,22699387
1600	12.276	2.594	4,73245952
3200	106.722	20.565	5,18949672

Concuerda con lo esperado, es más rápido la ejecución con la implementación en java ya que la los valores de los cocientes se aproximan al valor 5, que es mayor que 1.

Practica 2

TABLA 1= ALGORITMO BURBUJA

n	T ordenado	T inverso	T aleatorio
10.000	430	1.204	959
20.000	1.750	4.797	3.762
40.000	6.990	19.074	14.906
80.000	27.635	75.929	59.351
160.000	Fdt	Fdt	Fdt
320.000	Fdt	Fdt	Fdt
640.000	Fdt	Fdt	Fdt
1.280000	Fdt	Fdt	Fdt

Los resultados obtenidos del algoritmo burbuja para distintos n tiene sentido ya que es un algoritmo cuadrático que va ordenando a base de inspeccionar pares e intercambiar valores. Para todos los casos la complejidad será cuadrática, pero el tiempo de ejecución disminuye si el vector está ordenado porque no se hacen tantos intercambios y aumenta en el peor caso que sería un vector ordenado de manera inversa.

TABLA 2= ALGORITMO SELECCIÓN

n	T ordenado	T inverso	T aleatorio
10.000	396	344	395
20.000	1.613	1.382	1.580
40.000	6.428	5.632	6.415
80.000	25.779	22.619	25.429

160.000	102.155	89.164	101.770
320.000	Fdt	Fdt	Fdt
640.000	Fdt	Fdt	Fdt
1.280000	Fdt	Fdt	Fdt

Los resultados obtenidos del algoritmo Selección para los distintos valores de n concuerda con su complejidad cuadrática. Además, se aprecia que se comporta prácticamente igual para los tres casos ya que aunque al buscar el mínimo en los elementos no ordenados, no se encuentre ninguno, realiza igualmente un intercambio del pivote consigo mismo.

TABLA 3= ALGORITMO INSERCIÓN

n	T ordenado	T inverso	T aleatorio
10.000	1	803	401
20.000	1	3.192	1.566
40.000	1	12.713	6.284
80.000	2	50.459	25.480
160.000	4	Fdt	100.775
320.000	6	Fdt	Fdt
640.000	14	Fdt	Fdt
1.280000	27	Fdt	Fdt

Los resultados obtenidos del algoritmo de inserción concuerdan con lo esperado, en los casos de un vector ordenado de manera inversa y un ordenado aleatoriamente la complejidad será cuadrática, mientras que para un vector ordenado será lineal.

TABLA 4= ALGORITMO RÁPIDO

n	T ordenado	T inverso	T aleatorio
10.000	3	3	4
20.000	6	6	8
40.000	11	13	18
80.000	25	26	37
160.000	50	56	72
320.000	105	117	161
640.000	226	246	334
1.280000	466	515	701

Este algoritmo tiene una complejidad cuasi lineal por lo que los resultados obtenidos tienen sentido, el peor caso se daría si el elemento central fuera el mínimo o el máximo elemento del vector, lo que al parecer no sucede porque en ese caso la complejidad sería cuadrática.

Proyecte, a partir de las complejidades y los datos de las tablas anteriores, ¿cuántos días (1 día= 861400.000 milisegundos) tardaría cada uno de los cuatro métodos (Burbuja, Selección, Inserción y Rápido) en ordenar un vector de n= 81.920.000 (o
46
sea, n=80000*2 *2 =80000*2
10), en el caso inicialmente en orden aleatorio?
A partir de las complejidades y los datos de las tablas anteriores, calcular cuántos días tardaría cada método en ordenar un vector de n= 81.920.000 en el caso inicialmente aleatorio.
Sabiendo que $1\ día = 86.400.000ms$ y que para los tres primeros métodos la complejidad para dicho caso es cuadrática. Además de tener en cuenta que $81.920.000 = 80.000 * 2^{10}$, es decir, que estamos multiplicando el tamaño del problema en el caso para n = 80.000 por 2^{10}.

- Método Burbuja: multiplicamos T(80.000) por 1024^2 y pasamos a días → $\frac{59.351*1024^2}{86.400.000} = 720$ días
- Método Selección: multiplicamos T(80.000) por 1024^2 y pasamos a días → $\frac{25.429*1024^2}{86.400.000} = 309$ días
- Método Inserción: multiplicamos T(80.000) por 1024^2 y pasamos a días → $\frac{25.480*1024^2}{86.400.000} = 309$ días

El método rápido, por el contrario, tiene una complejidad cuasi lineal, por lo que solo tendremos que multiplicar por 1024

- Método Rápido: multiplicamos T(80.000) por 1024 y pasamos a días → $\frac{37*1024}{86.400.000} =$ 0,00043 días, es decir, 37 segundos.

TABLA 5= RÁPIDO vs. INSERCIÓN PARA TIEMPOS BAJOS

Repeticiones 10.000.

n	T rápido	T inserción	t rápido/t inserción
10	128	66	1,94
15	193	108	1,79
20	286	208	1,38
25	384	295	1,30
30	473	431	1,10
35	588	538	1,09
40	708	714	0,99
45	804	931	0,86
50	884	1.142	0,77
55	1000	1.363	0,73
60	1124	1.597	0,70
65	1197	1.815	0,66
70	1352	2.150	0,63
75	1425	2.413	0,59
80	1589	2.714	0,59
85	1676	3.090	0,54
90	1743	3.423	0,51
95	1954	3.745	0,52
100	2037	4.151	0,49

A partir del n = 40, el algoritmo rápido es mejor que el de inserción. Para valores de n pequeños tarda menos el de inserción por que tiene una implementación más sencilla, sin recursividad y demás.

Practica3

1)DyV por SUSTRACCIÓN

Analizando la complejidad de las clases Sutraccion1: O(n), Sustraccion2: O(n^2),

Sustraccion3:$O(2^n)$. Los tiempos coinciden ya que para el primero, al doblar n el tiempo también se dobla. Para el segundo, al doblar n, el tiempo se cuadriplica. Para el tercero, el tiempo se vuelve intratable imposible de medir para valores de n algo altos.
Las clases Sutraccion1 y Sustraccion2 dejan de dar valores de tiempo para el valor de n=8.000 porque da un StackOverFlow debido a que por culpa de la recursividad se llena la pila de llamadas.
Para calcular cuántos años tardaría en finalizar la ejecución Sustracción3 para n=80, escojo un n con un tiempo pequeño para comparar. Por ejemplo, n=20→T(20)=38ms.
Como este algoritmo tiene una complejidad de $O(2^n)$, significa que cada vez que aumenta en uno la n, el tiempo se duplica.

$$T(80) = 2^{80-20} * T(20) = 2^{60} * 38ms = 4{,}38 * 10^{19}ms = 4{,}38 * \frac{10^{19}}{31.536.000.000}$$
$$= 1{,}38\ mil\ millones\ de\ años\ (aprox)$$

Implementación de Sustraccion4 con complejidad $O(n^3)$:

```
public static void rec4 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=0;i<n;i++) cont++; // O(n)
        rec4 (n-1);
        for (int i=0;i<n;i++) cont++; // O(n)
    }
}
```

Tiempos ejecutando con nVeces =1:

N	100	200	400	800	16.00	32.000
T(ms)	17	16	125	982	51.470	Fdt

Implementación Sustraccion5 con $O\left(3^{\frac{n}{2}}\right)$:

```
public static void rec5 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        rec5 (n-2);
        rec5 (n-2);
        rec5 (n-2);
    }
}
```

Tiempos ejecutando con nVeces =1:

N	30	32	34	36	38	40
T(ms)	340	1.056	15.722	67.156	Fdt	Fdt

Usando la misma lógica que antes, Sustracción5 para n=80 tardaría:
Escogemos n=30 →T(30)=340ms

$$T(80) = 3^{\frac{80-30}{2}} * T(30) = 3^{25} * 340ms = \frac{2{,}88 * 10^{14}}{31.536.000.000} años = 9.134{,}9\ años\ (aprox)$$

2)DyV por DIVISIÓN

Calculamos las complejidades aplicando el teorema maestro: $T(n) = a * T\left(\frac{n}{b}\right) + O(n^k)$. El algoritmo de Divison1 tiene complejidad O(n) ya que: a=1, b=3,k=1; esto coincide con los tiempos obtenidos porque al doblar n también se dobla el tiempo. El algoritmo Division2 tiene una complejidad O(nlogn) ya que: a=2,b=2,k=1; esto coincide con los tiempos obtenidos ya que al doblar el valor de n los tiempos son aproximadamente el doble también. Por último, el

algoritmo Division3 tiene una complejidad O(n) ya que: a=2,b=2,k=0;esto coincide con los tiempos obtenidos, sucede igual que en el algoritmo 1.

Implementación de Division4 para que tenga una complejidad $O(n^2) con\ a < b^k$:

```
public static void rec4 (int n)
{
    if (n<=0)
        cont++;
    else
    {
        for (int i=1;i<n;i++) cont++ ;  //O(n)
        for (int i=1;i<n;i++){
            for (int j=1;j<n;j++){
                cont++ ;
            }
        }
        rec4 (n/2);
    }
}
```

Toma de tiempos con nVeces =1:

n	1000	2000	4000	8000	16000	32000	64.000
T(ms)	0	27	126	464	1935	68.562	Fdt

Implementación de Division5 para que tenga una complejidad $O(n^2) con\ a > b^k$:

```
public static void rec5 (int n)
    {
        if (n<=0)
            cont++;
        else
        {
            for (int i=1;i<n;i++) cont++ ;
            cont++ ; // O(1)
            rec5 (n/2);
            rec5 (n/2);
            rec5 (n/2);
            rec5 (n/2);
        }

    }
```

Toma de tiempos con nVeces =1:

n	1000	2000	4000	8000	16000	32000	64.000
T(ms)	29	125	473	1901	58802	Fdt	Fdt

3)DOS EJEMPLOS BÁSICOS

Dentro de la clase VectorSuma, vemos que se calcula la suma con tres métodos distintos:
Método1: este método es iterativo y tiene una complejidad O(n).
Método2: este método emplea recursividad con sustracción, su complejidad es lineal.
Método3: este método emplea recursividad con división, complejidad lineal.
El más eficiente será el metodo1 porque no sobrecarga la pila con llamadas, el menos eficiente será el método2 que para valores de n muy grandes daría StackOverFlow.
Toma de tiempos con nVeces = 1 para n=100

Método	Método1	Método2	Método3
T(nanosegundos)	406	1.454	7.902

Para la clase Fibonacci tenemos cuatro métodos distintos para resolver el mismo problema:
Método1: este método es iterativo y su complejidad es lineal.
Método2: este método es iterativo con el uso de un vector, su complejidad es lineal.
Método3: este método emplea recursividad de forma lineal, la complejidad será lineal.
Método4: este método emplea recursividad de forma doble, complejidad O(1.6**n)
Los métodos que menos tardaran, es decir, los más eficientes serán los tres primeros, siendo el más efectivo el primero. El menos efectivo será el cuarto método ya que realiza demasiadas llamadas recursivas hasta para valores de n bajos.
Toma de tiempos con nVeces = 1 para n=40

Método	Método1	Método2	Método3	Método4
T(nanosegundos)	912	1444	1985	1,36*10^10

4)PRÁCTICA DyV

Algoritmo PuntosTrivial:

```
double distanciaMinima = Double.MAX_VALUE;
Punto p1Min = null;
Punto p2Min = null;

for (int i = 0; i < puntos.size(); i++) {
    for (int j = i + 1; j < puntos.size(); j++) {

        Punto p1 = puntos.get(i);
        Punto p2 = puntos.get(j);

        double dx = p1.x - p2.x;
        double dy = p1.y - p2.y;
        double distancia = Math.sqrt(dx * dx + dy * dy);

        if (distancia < distanciaMinima) {
            distanciaMinima = distancia;
            p1Min = p1;
            p2Min = p2;
        }
    }
}
```

Tabla de tiempos al ejecutar PuntosTrivialTiempos.java sin optimización:

N	1024	2048	4096	8192	16384	32768	65536	131072
Tiempo(ms)	0	10	16	55	206	830	12461	69814

AlgoritmoPuntosDyv:

```java
static void dyv(List<Punto> puntos) {

    int n = puntos.size();

    int mitad = n / 2;
    List<Punto> izq = puntos.subList(fromIndex: 0, mitad);
    List<Punto> der = puntos.subList(mitad, n);

    dyv(izq);
    dyv(der);

    for (Punto p1 : izq) {
        for (Punto p2 : der) {
            double d = distancia(p1, p2);
            if (d < mejorDist) {
                mejorDist = d;
                mejorP1 = p1;
                mejorP2 = p2;
            }
        }
    }
}
```

Tabla de tiempos al ejecutar PuntosDyVTiempos.java sin optimización:

N	1024	2048	4096	8192	16384	32768	65536	131072
Tiempo(ms)	10	14	11	54	215	886	13248	86541